DEPARTMENT OF MECHATRONICS

MECHATRONIC SYSTEM INTEGRATION

(MCTA 3203)

SEMESTER 1, 2026/2027

EXPERIMENT 2

SERIAL COMMUNICATION

SECTION 1

GROUP 7

No.	Matric No.	Name
1	2412749	MUHAMMAD ZUHAIL BIN MOHAMED ZUKILAN
2	2419147	MUHAMMAD SYAZWAN BIN SHOIB
3	2412308	NUR EZZATUL SYUHADA BINTI NOR AZIZI
4	2411522	NUR SHAZMIERA BINTI ZUBIR

Date of Submission : 18 October 2026

TABLE OF CONTENT

ABSTRACT 3
PART 1:READING POTENTIOMETER VALUES VIA SERIAL COMMUNICATION 3
1.0 INTRODUCTION AND THEORY 3
1.1 MATERIALS AND EQUIPMENTS 4
1.2 EXPERIMENTAL SETUP AND HARDWARE ASSEMBLY 4
1.3 METHODOLOGY AND PROGRAM LOGIC 4
1.4 DATA COLLECTION AND TECHNICAL ANALYSIS 6
1.5 RESULTS AND ENGINEERING DISCUSSION 7
1.6 CONCLUSION 8
PART 2: CONTROLLING SERVO MOTOR VIA SERIAL COMMUNICATION 9
2.0 INTRODUCTION AND THEORY 9
2.1 MATERIALS AND EQUIPMENT 9
2.2 EXPERIMENTAL SETUP AND HARDWARE ASSEMBLY 10
2.3 METHODOLOGY AND PROGRAM LOGIC 10
2.4 DATA COLLECTION AND TECHNICAL ANALYSIS 13
2.5 RESULTS AND ENGINEERING DISCUSSION 15
2.6 CONCLUSION 16
RECOMMENDATIONS 16
REFERRENCES 17
ACKNOWLEDGEMENT 17
STUDENT DECLARATION 17

ABSTRACT

This experiment aimed to investigate serial communication between an Arduino microcontroller and a computer using Python for sensor data monitoring and actuator control. The hardware used included an Arduino board, potentiometer, servo motor, LED, 220 Ω resistor, and connecting wires. In the main experiments, potentiometer readings were transmitted from the Arduino to Python through USB serial communication, while servo motor angles were controlled using commands sent from Python to the Arduino.

For Task 1, an LED was added to the circuit and programmed to turn on when the potentiometer reading exceeded half of its maximum range and turn off otherwise. For Task 2, the potentiometer was integrated to adjust the servo motor angle in real time, with an additional keyboard command to stop program execution. Servo position data was received through the serial connection, and Matplotlib was used to plot potentiometer readings in real time. The PySerial library enabled communication between the Arduino and Python at a baud rate of 9600 bps. The expected results included potentiometer readings within the Arduino's 10-bit analog range of 0–1023, correct LED switching based on the programmed threshold, and servo movement within the range of 0° to 180°. However, potential challenges included mismatched baud rates, and the Serial Monitor was not closed, preventing the Python code in PyCharm from running.

It is recommended to verify circuit connections, maintain consistent serial settings, and test each function individually before integrating the complete system to improve reliability and performance.

PART 1:READING POTENTIOMETER VALUES VIA SERIAL COMMUNICATION

1.0 INTRODUCTION AND THEORY

A potentiometer is an analog input component that produces variable voltage signals when its knob is rotated. The Arduino reads these signals through analog pin A0 and converts them into digital values ranging from 0 to 1023 using its 10-bit Analog-to-Digital Converter (ADC), with a 5 V reference voltage. An LED connected in series with a 220 Ω resistor acts as an output indicator while the resistor limits the current to protect the LED.

The objective of this experiment is to transmit potentiometer readings from Arduino to Python via USB serial communication. The experiment aims to display and visualize the readings in real time and control an LED based on a predefined threshold. The expected outcomes are successful data transmission, accurate monitoring of potentiometer values and automatic LED operation. The LED should turn on when the reading exceeds approximately 511 and turn off otherwise.

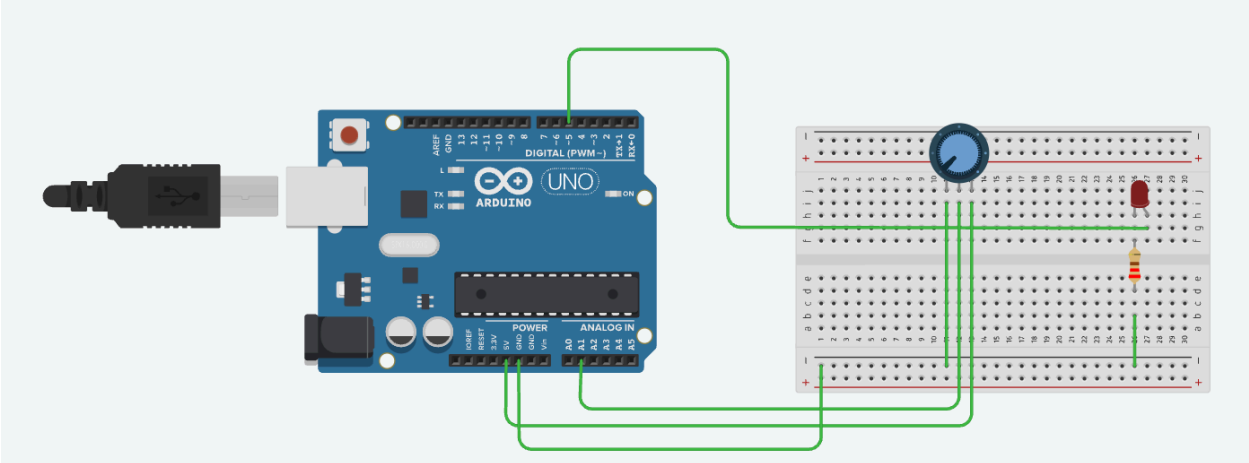
The Arduino is programmed using the Arduino IDE to continuously read the potentiometer values and transmit them through USB serial communication at a specified baud rate such as 9600 bits per second. On the computer side, a Python script uses the PySerial library to receive and display the transmitted data through the correct COM port. The readings can also be visualized using the Arduino IDE Serial Plotter. Python processes the received values using conditional statements to determine whether the LED should be switched on or off. For reliable communication, the baud rate must match between both programs and the Serial Plotter must be closed when Python is accessing the same serial port.

1.1 MATERIALS AND EQUIPMENTS

The experiment was conducted using the following components:

- Arduino board
- Potentiometer
- Jumper wires
- Breadboard
- 220 Ω resistor
- LED
- Arduino IDE
- Pycharm

1.2 EXPERIMENTAL SETUP AND HARDWARE ASSEMBLY



1. The potentiometer was connected to the pin A0, ground and 5V.
2. The LED was connected in series with 220 Ω resistor to pin 5.

1.3 METHODOLOGY AND PROGRAM LOGIC

Experiment steps:

1. The Arduino was connected to the computer in the diagram above.

2. Upload the code uploaded to Arduino and the Python ran.
3. As the potentiometer knob turned, the potentiometer readings displayed in the Python terminal.
4. The circuit was being extended by adding LED in series with a 220 resistor.
5. The coding uploaded such that the potentiometer value exceeded half of its maximum range, the LED turned on, otherwise turned off.

IDE code:

```
const int portLED = 5;

void setup() {
  pinMode(portLED, OUTPUT);
  Serial.begin(9600);
}
void loop() {
int potValue = analogRead(A0);
if(potValue>511){
  digitalWrite(portLED, HIGH);
}
else{
  digitalWrite(portLED, LOW);
}
Serial.println(potValue);
delay(1000);
}
```

Python code:

```
import serial

ser = serial.Serial('COM3', 9600)

try:
    while True:
        line = ser.readline().decode().strip()
        if line.isdigit():
            pot_value = int(line)
            print("Potentiometer Value:", pot_value)

            if pot_value > 511:
```

```
            ser.write(b'H')  # Send 'H' to turn LED ON
        else:
            ser.write(b'L')  # Send 'L' to turn LED OFF

except KeyboardInterrupt:
    ser.close()
    print("Serial connection closed.")
```

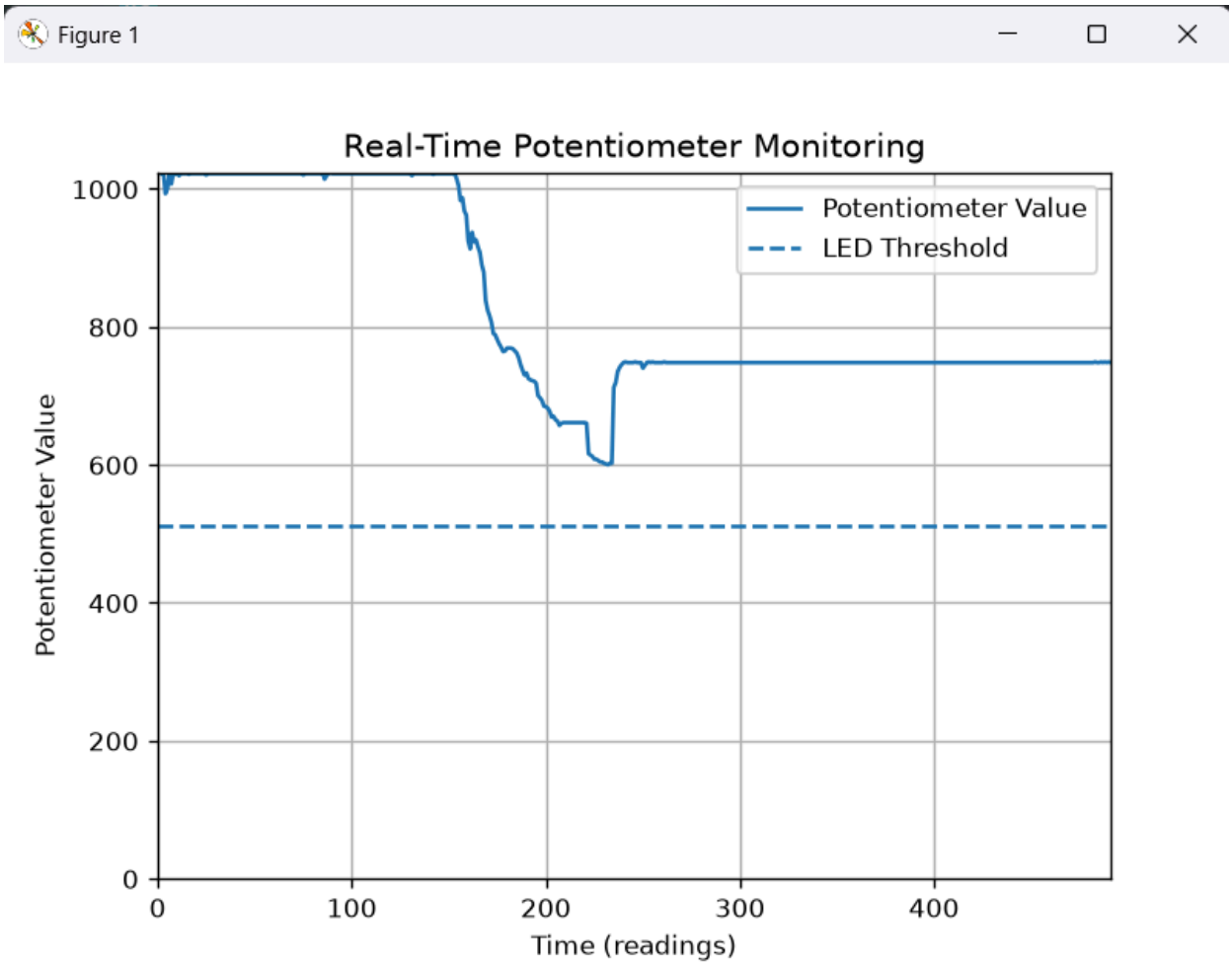
1.4 DATA COLLECTION AND TECHNICAL ANALYSIS

The data was collected by observing the resistance value displayed on the serial monitor where controlled by the potentiometer. If the resistance value is higher than half of maximum or threshold value which is 511 Ω, the LED turns on and otherwise will be turned off. The observed results are presented in the tables below.

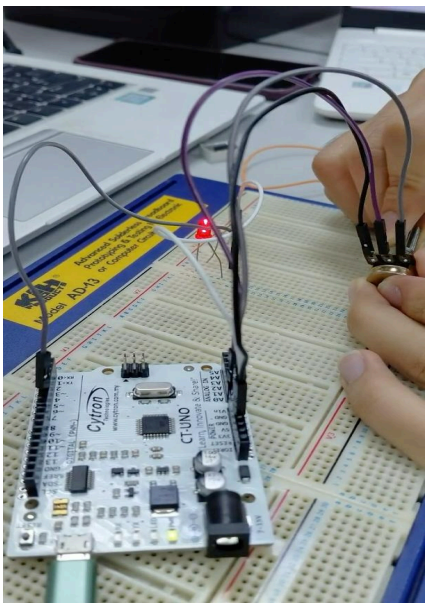
Potentiometer Value	LED
0	Off
256	Off
512(Threshold value)	On
1023	On

Table 1: Relationship Between Potentiometer Values and LED Status

1.5 RESULTS AND ENGINEERING DISCUSSION







As the potentiometer was adjusted, the displayed readings changed accordingly, demonstrating that the Arduino could detect variations in the analog input voltage and transmit the corresponding digital values. The Serial Plotter provided a graphical representation of the potentiometer readings in real time. The graph showed changes in the readings as the potentiometer knob was rotated, allowing the relationship between the potentiometer's position and its output values to be observed more clearly. The readings displayed in the Serial Monitor were consistent with the values represented in the graph, indicating that the data transmission and visualization were functioning as intended.

1.6 CONCLUSION

The experiment successfully demonstrates the practical application of analog input, analog-to-digital conversion, serial communication, and real-time data visualization. The potentiometer connected to analog pin A0 produced digital readings ranging from 0 to 1023, depending on the input voltage. The readings were displayed in real time through the Serial Monitor and Serial Plotter, allowing changes in the potentiometer values to be observed and analyzed. These techniques are important in engineering systems involving sensors, where physical changes must be converted into digital data for monitoring, analysis, and further processing.

PART 2: CONTROLLING SERVO MOTOR VIA SERIAL COMMUNICATION

2.0 INTRODUCTION AND THEORY

A potentiometer is an analog input component that produces variable voltage signals when its knob is rotated. The Arduino converts these signals into digital values ranging from 0 to 1023 using its 10-bit Analog-to-Digital Converter (ADC). These readings are used to determine the angle of a servo motor which operates by receiving control signals to rotate to a specific position within its operating range. An LED can also be incorporated as a visual indicator to represent a particular potentiometer value or servo position.

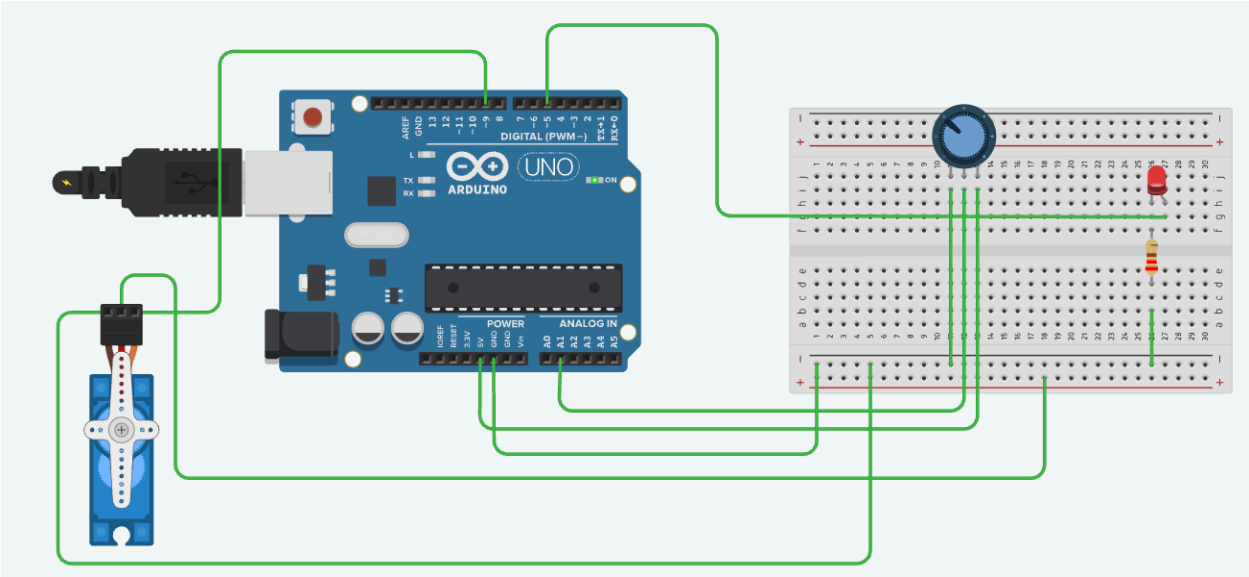
The objective of this experiment is to control the servo motor angle using potentiometer readings transmitted between Arduino and Python, visualize sensor data graphically, and implement additional LED feedback. The expected outcomes are real-time servo movement corresponding to potentiometer adjustments, successful display of servo position data and real-time graphical visualization of potentiometer readings. The LED is expected to switch on or off according to a predefined threshold while the system should also support stopping execution using a designated keyboard key.

The Arduino is programmed using the Arduino IDE and the Servo library to read potentiometer values and control the servo motor. USB serial communication enables data exchange between the Arduino and a Python application running on the computer. The Python script uses the PySerial library to receive and display servo position data, while the Matplotlib library is used to plot potentiometer readings in real time. Conditional statements can be implemented to control the LED based on the potentiometer value or servo angle while keyboard ‘q’ input allows the user to stop the program. These components demonstrate the integration of sensor input, actuator control, serial communication and graphical data visualization in a mechatronics system.

2.1 MATERIALS AND EQUIPMENT

- Arduino board + USB cable
- Servo motor
- Jumper wires
- Potentiometer
- LED
- Arduino IDE
- Pycharm

2.2 EXPERIMENTAL SETUP AND HARDWARE ASSEMBLY



1. The potentiometer was connected to pin A0, ground and 5V.
2. The servo motor was connected to pin 9, ground and 5V.
3. The LED was connected in series with 220 Ω resistor to pin 5.

2.3 METHODOLOGY AND PROGRAM LOGIC

Experiment steps:
IDE code:

```
#include <Servo.h>

Servo myservo;
const int potPin = A0;
const int servoPin = 9;
const int ledPin = 5;

void setup() {
  Serial.begin(9600);
  while (!Serial);

  myservo.attach(servoPin);
  pinMode(ledPin, OUTPUT);
}

void loop() {
  if (Serial.available() > 0) {
```

```
    char command = Serial.read();

    if (command == 'q') {
      digitalWrite(ledPin, LOW);
      myservo.write(90);

      while (Serial.available() > 0) {
        Serial.read();
      }

      Serial.println("SYSTEM RESET ACKNOWLEDGED");
      delay(500);
      return;
    }
  }

  int potValue = analogRead(potPin);

  int servoPos = map(potValue, 0, 1023, 0, 180);

  myservo.write(servoPos);

  if (potValue > 511) {
    digitalWrite(ledPin, HIGH);
  } else {
    digitalWrite(ledPin, LOW);
  }

  Serial.print("Potentiometer: ");
  Serial.print(potValue);
  Serial.print(" | Servo Angle: ");
  Serial.println(servoPos);

  delay(50);
}
```

Python code:

```
import serial
import matplotlib.pyplot as plt
import matplotlib.animation as animation
```

```
import re

PORT = 'COM3'
BAUD = 9600
MAX_POINTS = 50  # Number of data points to show on screen at once

try:
   ser = serial.Serial(PORT, BAUD, timeout=0.1)
except Exception as e:
   print(f"Could not open serial port {PORT}: {e}")
   exit()

x_data = []
y_data = []
counter = 0

fig, ax = plt.subplots(figsize=(8, 4))
line, = ax.plot([], [], 'b-', lw=2, label="Potentiometer Value")

ax.set_xlim(0, MAX_POINTS)
ax.set_ylim(0, 1025)  # Potentiometer ranges from 0 to 1023
ax.set_title("Real-Time Arduino Potentiometer Monitor\n(Press 'q' on this
window to reset & exit)")
ax.set_xlabel("Sample Index")
ax.set_ylabel("Value (0-1023)")
ax.grid(True)
ax.legend(loc="upper right")


def init():
   line.set_data([], [])
   return line,


def update(frame):
   global counter
   if ser.in_waiting > 0:
       try:
           line_str = ser.readline().decode('utf-8').strip()
           match = re.search(r'Potentiometer:\s*(\d+)', line_str)
           if match:
               pot_val = int(match.group(1))

               x_data.append(counter)
               y_data.append(pot_val)
               counter += 1

               # Keep window rolling by slicing the data lists
               if len(x_data) > MAX_POINTS:
```

```
                    x_data.pop(0)
                    y_data.pop(0)
                    ax.set_xlim(x_data[0], x_data[-1])
                else:
                    ax.set_xlim(0, MAX_POINTS)

                line.set_data(x_data, y_data)
        except Exception as err:
            print(f"Parse error: {err}")

    return line,


def on_key(event):
    if event.key in ['q', 'Q']:
        print("\n[INFO] 'q' pressed. Sending reset command to Arduino...")
        try:
            ser.write(b'q')
        except Exception as e:
            print(f"Error writing to serial: {e}")
        ser.close()
        plt.close()


fig.canvas.mpl_connect('key_press_event', on_key)

ani = animation.FuncAnimation(
    fig, update, init_func=init, interval=50, blit=True, cache_frame_data=False
)

plt.show()

if ser.is_open:
    ser.close()
    print("[INFO] Serial port closed.")[]
```

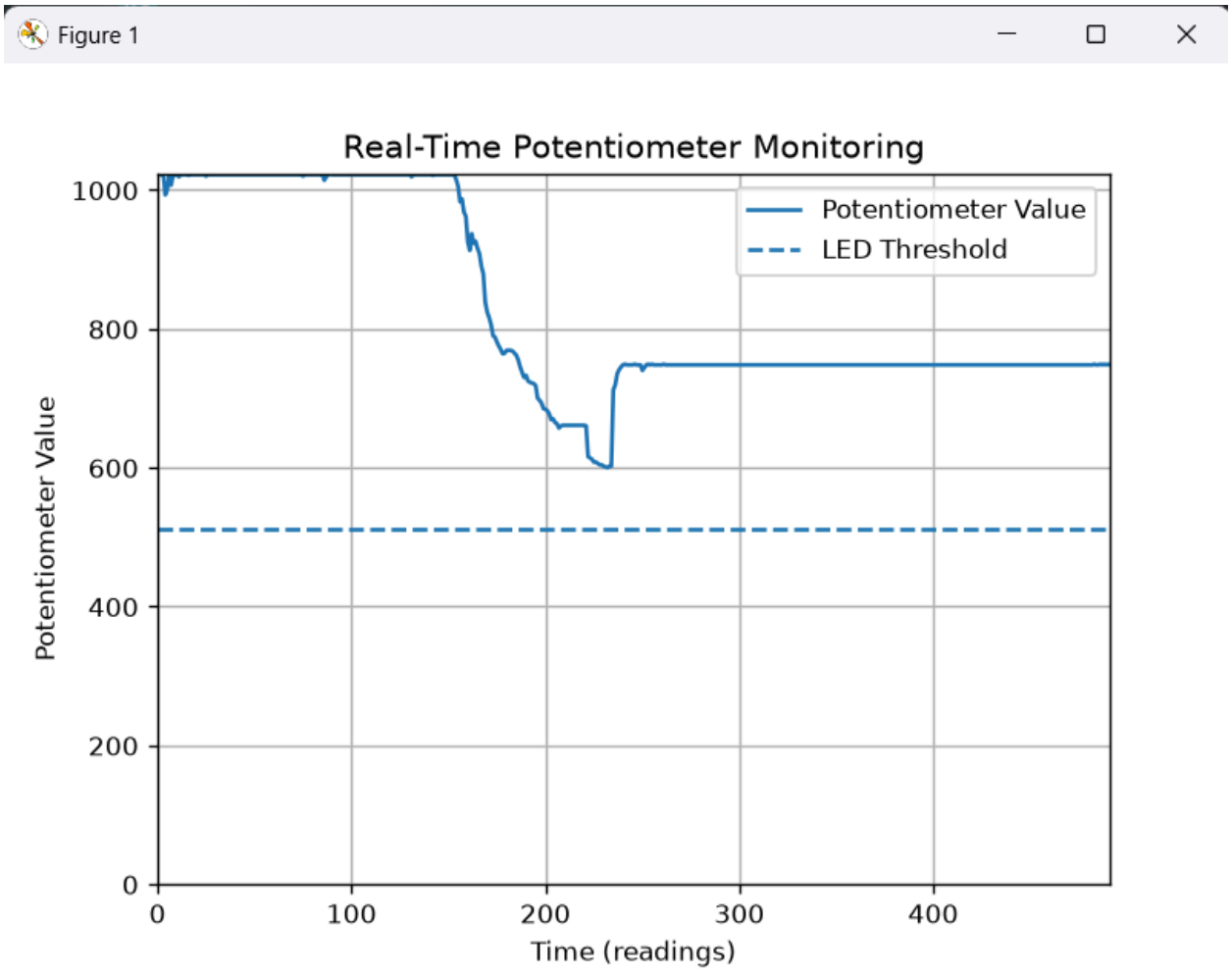
2.4 DATA COLLECTION AND TECHNICAL ANALYSIS

The data was collected by observing the resistance value displayed on the serial monitor and servo position data where controlled by the potentiometer. If the resistance value is higher than half of maximum value which is 511, the LED is turned on and otherwise will be turned off while the position of servo is limited from 0 to 180. The observed results are presented in the tables below.

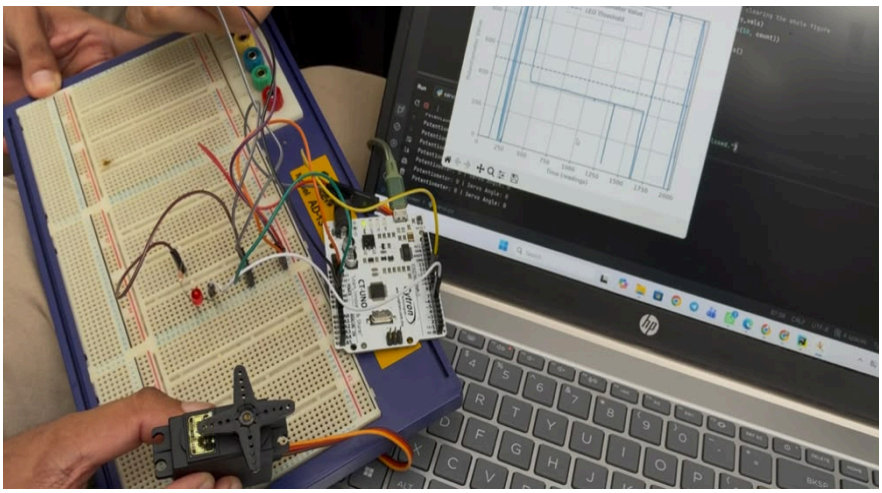
Potentiometer Value	Servo Angle(Blade Rotation)	LED
0	0°	Off
256	45°	Off
512(Threshold value)	90°	On
1023	180°	On

Table 2: Relationship Between Potentiometer Values,Servo Angle and LED Status

2.5 RESULTS AND ENGINEERING DISCUSSION







Potentiometer values plotted in real-time by integrating graphical visualization using Matplotlib. As shown in the graph, the potentiometer values displayed in the serial monitor are consistent with those represented in the real time graph. This allows changes in the potentiometer readings to be monitored continuously and provides a clearer understanding of the system's response.

2.6 CONCLUSION

The experiment successfully demonstrates servo motor control using serial communication between an Arduino microcontroller and Python via USB at a baud rate of 9600 bps. The servo motor can rotate to the desired angle based on the control commands, while the PySerial library facilitates data exchange between the Arduino and Python. The servo angle data is also visualized in real time using the Matplotlib library, allowing the system's response to be monitored continuously. In addition, the program can be terminated by pressing the 'q' key. Overall, the experiment achieves its objective of demonstrating serial communication, servo motor operation, and real-time data visualization, which are essential concepts in the development of engineering control systems.

RECOMMENDATIONS

For Task 1, signal filtering is recommended to improve the stability and accuracy of potentiometer readings. Small mechanical movements and electrical noise may cause fluctuations in the analog-to-digital converter (ADC) values, resulting in the LED switching on and off when the readings are near the threshold. Applying a moving average filter can reduce these fluctuations and provide more stable LED operation.

Furthermore, serial communication between the Arduino and Python can be improved by ensuring that the correct COM port and baud rate are selected. The Serial Plotter should also be closed when running the Python script to prevent conflicts when accessing the serial port.

For Task 2, a moving average or low-pass filter can be implemented to reduce noise in the potentiometer readings and improve the stability of real-time data visualisation. Additionally, the servo motor movement can be made smoother by applying gradual angle adjustments instead of directly changing to the target angle. These improvements can enhance servo control, reduce sudden movements, and improve the overall reliability of the system.

REFERRENCES

https://youtu.be/V63frQDl2EU?si=l-nkiPzg2aHMNRBm

https://classroom.google.com/c/NjEzNDM1MTEzNDI2

ACKNOWLEDGEMENT

We would like to express our deepest appreciation to our instructor and lab personnel for their guidance and support throughout this experiment.

STUDENT DECLARATION

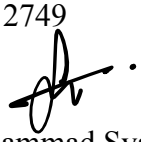
We hereby declare that the work presented in this technical report is entirely our own, except where explicitly acknowledged. We certify that in developing and completing this mechatronics project, we have strictly adhered to the standards of academic integrity and have not engaged in any form of plagiarism or unethical conduct. All sources of information, reference materials, and technical support utilized in this work have been appropriately cited and acknowledged.

Certificate of Originality and Authenticity

This is to certify that we are responsible for the work submitted in this report, that the original work is our own except as specified in the references and acknowledgment, and that the original work contained herein has not been undertaken or done by unspecified sources or persons. We hereby certify that this report has not been done by only one individual, and all of us have contributed to the report. The length of contribution to the reports by each individual is noted within this certificate. We also hereby certify that we have read and understand the content of the total report, and no further improvement on the reports is needed from any of the individual contributors to the report. We therefore agreed unanimously that this report shall be submitted for marking, and this final printed report has been verified by us.

Signature: 
Name: Muhammad Zuhail Bin Mohamed Zukilan
Matric : 2412749

Read[/]
Understand[/]
Agree[/]

Signature: 
Name: Muhammad Syazwan Bin Shoib
Matric : 2419147

Read[/]
Understand[/]
Agree[/]

Signature: 
Name: Nur Ezzatul Syuhada Binti Nor Azizi
Matric : 2412308

Read[/]
Understand[/]
Agree[/]

Signature: 
Name: Nur Shazmiera Binti Zubir
Matric : 2411522

Read[/]
Understand[/]
Agree[/]